Part 3: **Client-Side Programming**

Programming Webpages

- Why?
  - Make webpages interactive (ex: menus, side bars, games)
  - Dynamically insert/modify elements (ex: context information)
  - React to events (ex: page load, user clicks)
  - Get information about a user's computer (ex: browser type)
  - Perform calculations on user's computer (ex: form validation)
  - Create more complicated responsive pages
  - Asynchronous communication with server to load/push data
  - …
- How?
  - Add JavaScript code into webpages!
  - JavaScript is the only standard programming language to browsers
    - Make sure to get away from obsolete technologies like VBScript, Flash, Silverlight,…

Including Scripts into Webpages

- There are 3 methods:
  - Embedded code
    - ```
      <script>
      const a = document.getElementById("demo");
      a.innerHTML = "Hello";
      </script>
      ```
  - External code
    - ```
      <script src="my-script.js"></script>
      ```
    - ```
      <script src="https://remote/script.js"></script>
      ```
  - Inline code (to be avoided - explanation comes later)
    - ```
      <button onclick="toggleMenu()">Menu</button>
      ```
    - ```
      <img onerror="console.log('error'); hide(this);"
        src="https://unstable-server/some-image.jpg" />
      ```
      ```

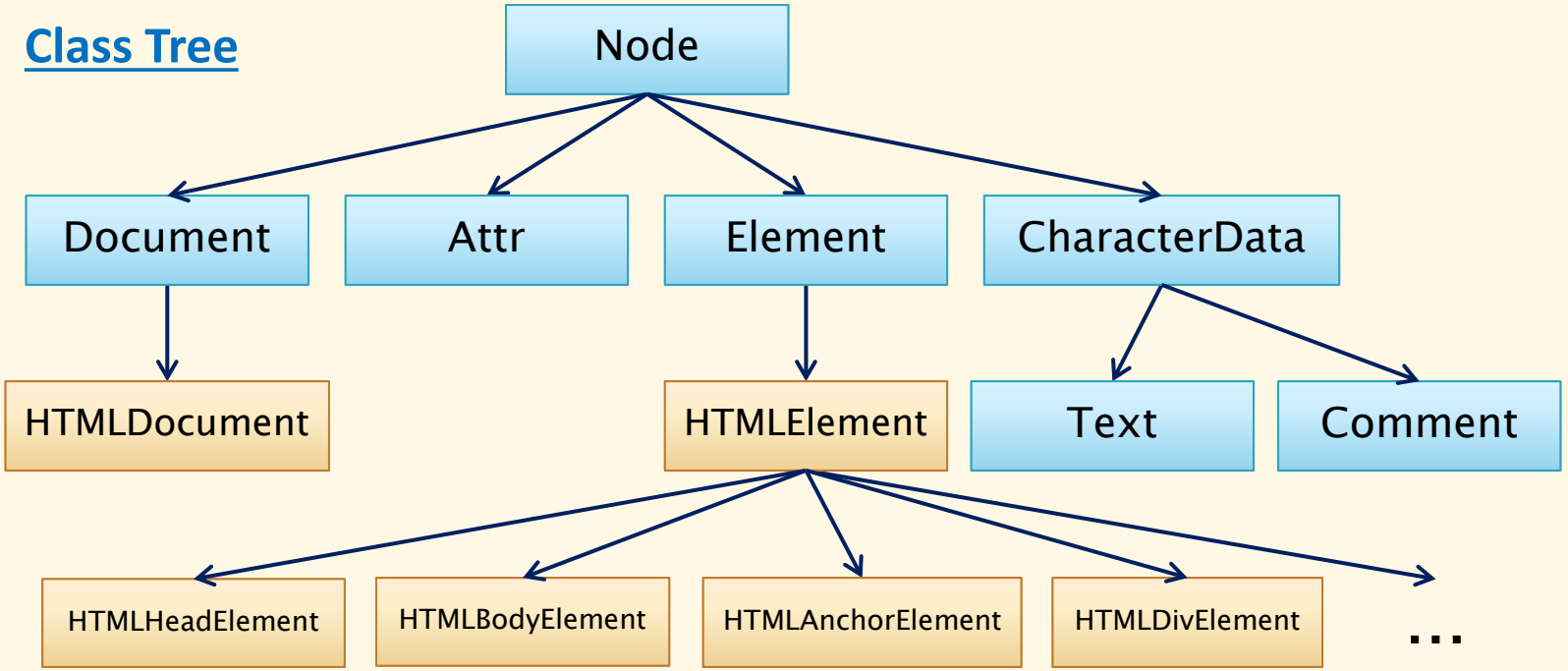
Notes on `type` Attribute

- `<script type="text/javascript">`
  - Historically used, but obsoleted
  - Still supported by most browsers
- `<script type="application/javascript">`
  - New official MIME type
  - Some old browsers may not recognize
- `<script>`
  - Let the browser choose its default value
  - Recommended!

DOM Manipulation

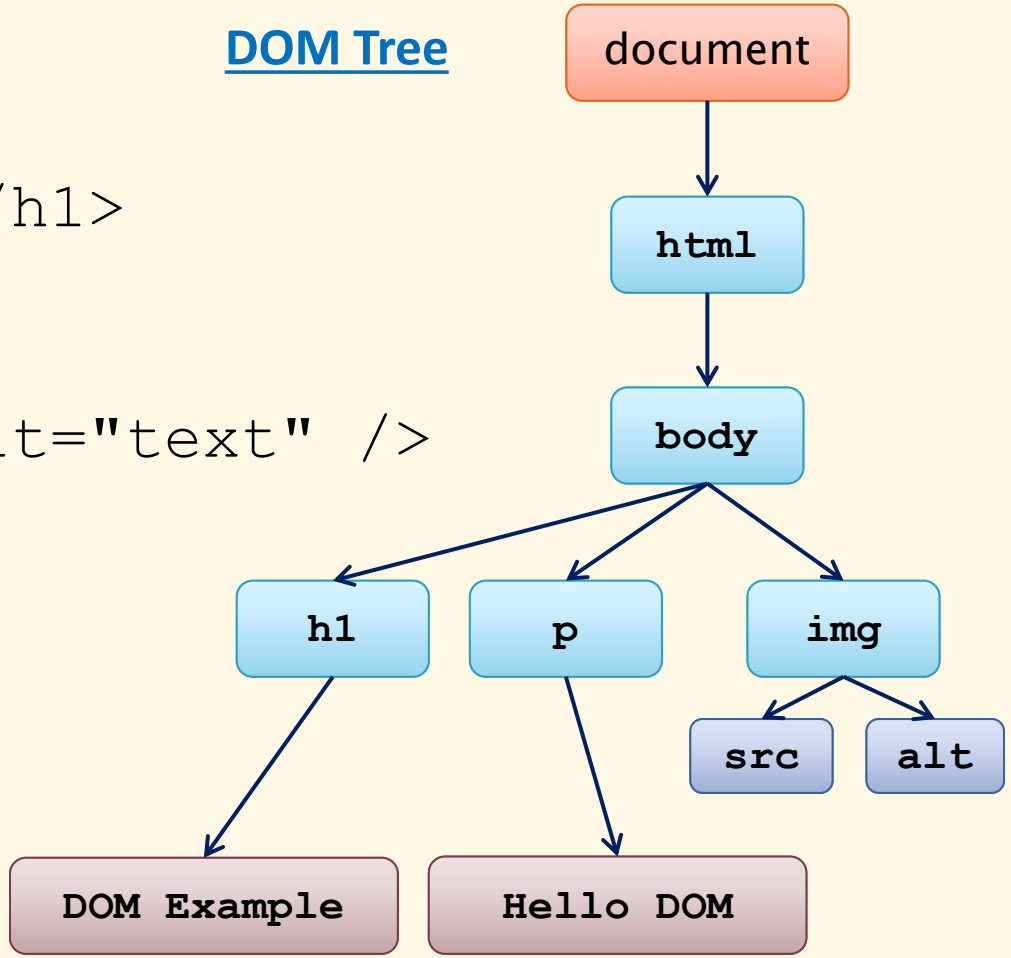
DOM: Document Object Model

- Object model representing HTML/XML tree
- Class of each node corresponds with the node type
- Different nodes allow different methods

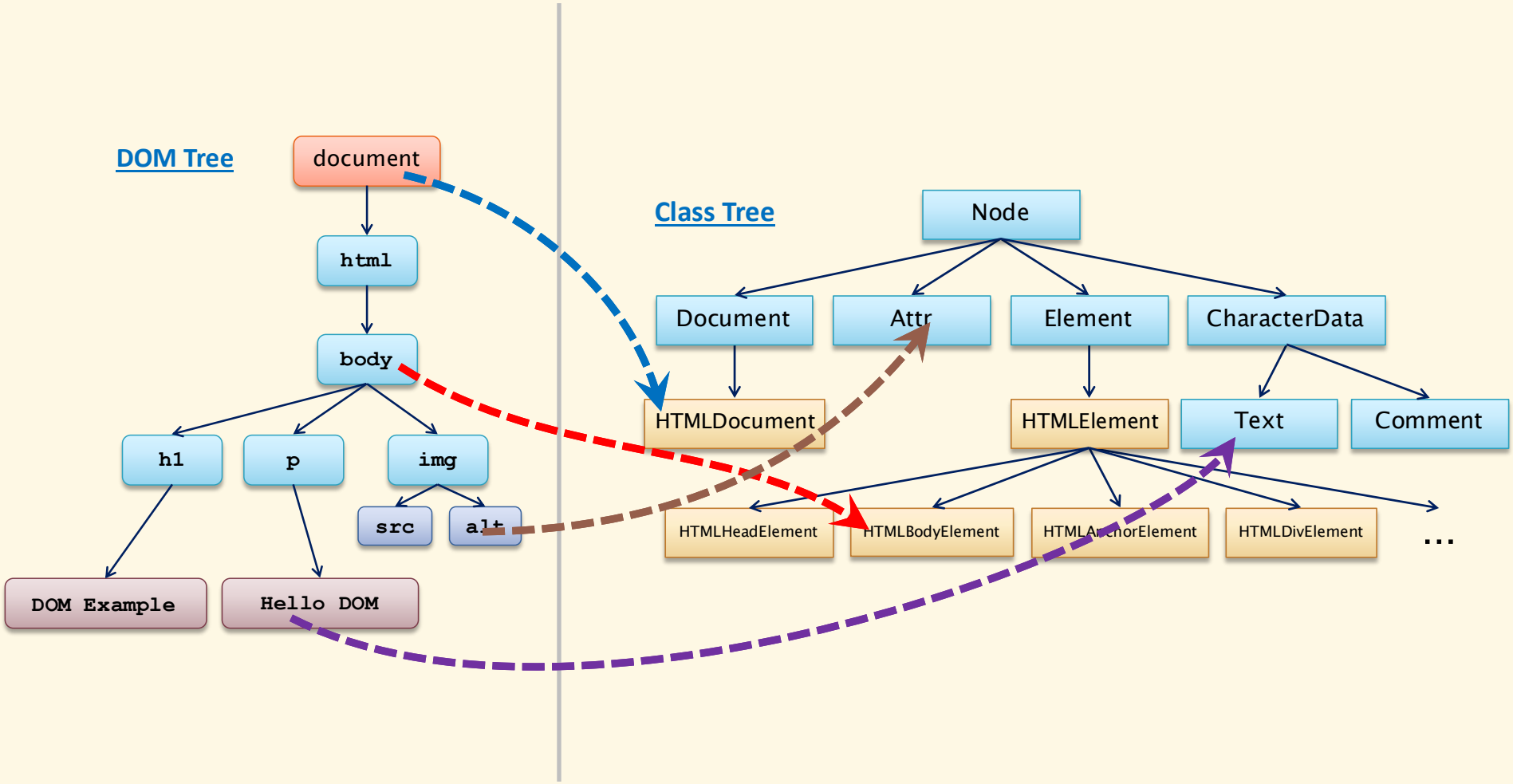


Example

```
<html>
   <body>
      <h1>DOM Example</h1>
      <p>Hello DOM</p>
      <img src="url" alt="text" />
   </body>
</html>
```



Object-to-Class Mapping



DOM Traversal: `Node` Class

- `firstChild`, `lastChild`: first/last child node (including comment and text nodes)
- `childNodes`: live collection of child nodes
- `nextSibling`, `previousSibling`: next/previous sibling
- `parentNode`: parent node
- `parentElement`: parent element

- Live collection:
  - Changes in the DOM automatically update the collection

DOM Traversal: `Document` Class

- `documentElement`: document's root element
- `head`, `body`: document's `<head>` and `<body>` elements
- `getElementById(`*`id`*`)` : returns element for given ID
- `getElementsByName(`*`name`*`)` : returns live collection of elements for given name
- `getElementsByClassName(`*`class`*`)` : returns live collection of elements for given class name
- `getElementsByTagName(`*`tag`*`)` : returns live collection of elements for given tag name
- `querySelector(`*`sel`*`)` : returns first element that matches the selector(s)
- `querySelectorAll(`*`sel`*`)` : returns static list of elements that match the selector(s)

DOM Traversal: `Element` Class

- `firstElementChild`, `lastElementChild`: first/last child elements (excluding comment and text nodes)
- `children`: live collection of child elements
- `nextElementSibling`, `previousElementSibling`: next/previous sibling
- `getElementById(`*`id`*`)`
- `getElementsByName(`*`name`*`)`
- `getElementsByClassName(`*`class`*`)`
- `getElementsByTagName(`*`tag`*`)`
- `querySelector(`*`sel`*`)`
- `querySelectorAll(`*`sel`*`)`

Useful Attributes

- `Element` class:
  - `tagName`: element's tag name
  - `id`: element's identifier
  - `outerHTML`, `innerHTML`: HTML content of the element and/or its descendants
  - `textContent`: text content of its descendants, including hidden elements

- `HTMLElement` class:
  - `outerText`, `innerText`: text content of the element and/or its descendants, not including hidden elements

Exercise

- Write a code snippet to update the text of a given <p> element every second with the current time, using the format in the following example:
  - 2024-04-05 08:33:20

Position and Size Attributes

- `Element` class:
  - `clientLeft`, `clientTop`, `clientWidth`, `clientHeight`: content area + padding – scroll area
  - `scrollLeft`, `scrollTop`, `scrollWidth`, `scrollHeight`: scroll area

- `HTMLElement` class:
  - `offsetLeft`, `offsetTop`, `offsetWidth`, `offsetHeight`: relative position/size of the outer border of the current element to the inner border of the `offsetParent` node

Creating Elements and Text Nodes

```
const image = document.createElement("img");
image.src = "picture.jpg";
image.style = "width:2rem; max-width:100%";
document.body.appendChild(image);
```

```
const text = document.createTextNode("Content");
div.appendChild(text);
```

```
const div = document.getElementById("div1");
div.innerHTML = "<h1>Title</h1> <p>Content</p>";
```

```
const tab = document.getElementsByTagName("table")[0];
const tab2 = tab.cloneNode(true);
document.body.appendChild(tab2);
```

Node Manipulation

- `Element.prepend(elms)`: adds child element(s) at the beginning
- `Node.appendChild(elm)`, `Element.append(elms)`: adds child element(s) at the end
- `Node.insertBefore(elm, refNode)`: adds element before a reference child node
- `Node.removeChild(elm)`: removes a child
- `Node.replaceChild(new, old)`: replaces a child by another
- `Element.replaceChildren(new)`: replaces child elements by others

- How about insert after?
  - `parent.insertBefore(elm, refNode.nextSibling);`

Working with Attributes

- `Element.attributes`
- `Element.getAttribute(`*`attr`*`)`
- `Element.setAttribute(`*`attr`*`, `*`value`*`)`
- `Element.hasAttribute(`*`attr`*`)`
- `Element.removeAttribute(`*`attr`*`)`

- Examples:
  - `image.src = "picture.png";` (standard attributes only)
  - `image.setAttribute("src", "picture.png");`
  - `image.setAttribute("data-custom", "Nice day!");`

Working with Classes

- `Element.setAttribute("class", classes)`
- `Element.className`: string representing the space-separated class(es) of the element
- `Element.classList`: list of class(es)
  - `.contains(class)`
  - `.add(class)`
  - `.remove(class)`
  - `.replace(old, new)`
  - `.item(i)`

Working with Styles

- Element styles
  - `Element.setAttribute("style", value)`
  - `HTMLElement.style`
    - `.getPropertyValue(propName)`
    - `.removeProperty(propName)`
    - `.setProperty(propName, value)`
- Global styles
  - `Document.styleSheets`
    - `.deleteRule(idx)`
      `.insertRule(rule, idx)`
  - `Document.styleSheets[sIdx].cssRules[rIdx]`
    - `.selectorText`: CSS selector of the rule
    - `.style`: same as `Element.style`

Style Property Naming

CSS property (kebab-case)	`style` object property (camelCase)
color	color
padding	padding
background-color	backgroundColor
border-top-width	borderTopWidth
font-size	fontSize
font-family	fontFamily
…	…

- `<div style="font-size: 20pt">...</div>`
- `div.style.fontSize = "20pt";`

Examples

- Modify element style:

```
const div = document.getElementById("title");
div.style.removeProperty("border");
div.style.backgroundColor = "#f07";
```

- Add a rule to a global stylesheet:

```
const sheet = document.styleSheets[0];
sheet.insertRule(".warning {color: #ff3}", 0);
```

- Modify style properties of a rule in a global stylesheet:

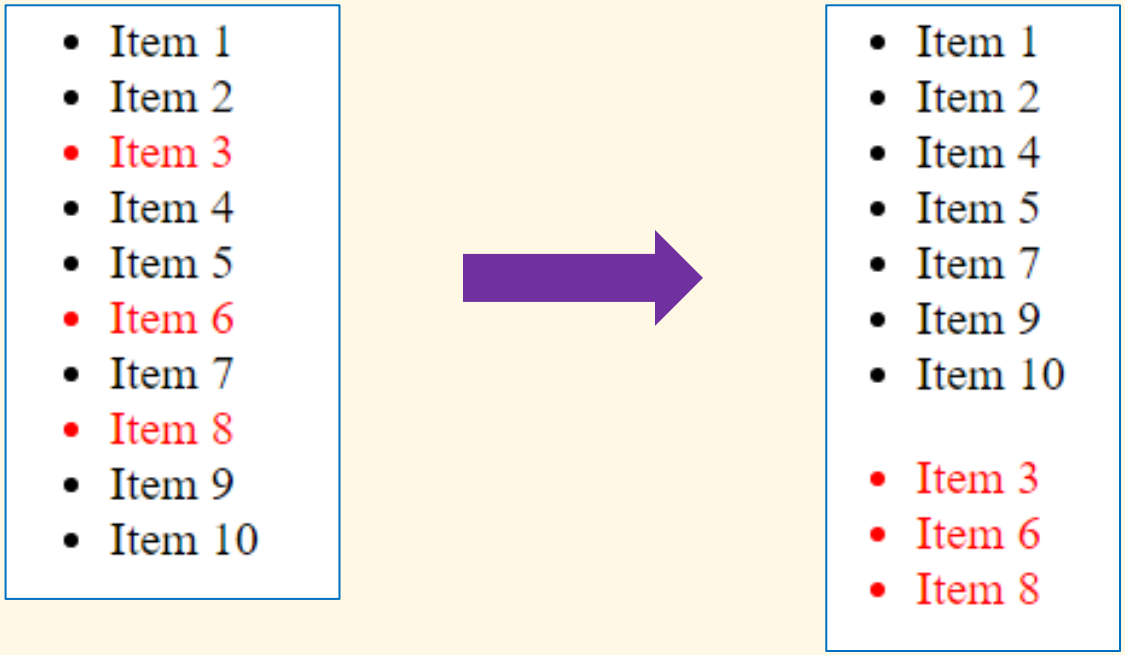
```
const rule = document.styleSheets[1].cssRules[5];
rule.style.setProperty("width", "100%");
rule.style.backgroundColor = "#f07";
```

- Create a new stylesheet with rules:

```
const style = document.createElement("style");
const rules = "#bold {font-weight: bold}";
style.appendChild(document.createTextNode(rules));
document.head.appendChild(style);
```

Exercises

1. Sort items of a list by the displayed text
2. Iterate an `<ul>` list, find all items in red and move them to a newly created list



Data Attributes

Introduction

- HTML5 allows data to be associated with elements using `data-*` attributes
- This is useful in writing CSS rules, and manipulating the DOM elements with JavaScript
- Define data attributes
  - ```
    <article class="car" data-cols="3" data-id="123">
      ...
    </article>
    ```
  - ```
    <button data-action="expand" data-target="dialog">
      {}
    </button>
    <button data-action="close" data-target="dialog">
      x
    </button>
    ```

CSS Access

- Like other normal HTML attributes
- Examples:
  - ```
    article::before {
      content: attr(data-parent);
    }
    ```
  - ```
    article[data-cols='3'] {
      width: 400px;
    }
    article[data-cols='4'] {
      width: 600px;
    }
    ```

JavaScript Access

- Using `getAttribute()`, `setAttribute()` like normal attributes
  - ```
    const a = document.querySelector(".car");
    const cols = a.getAttribute("data-cols");
    a.setAttribute("data-cols", cols + 1);
    ```

- Using `dataset` property
  - ```
    const a = document.querySelector(".car");
    const cols = a.dataset.cols;
    a.dataset.cols = cols + 1;
    ```

Exercise

- Consider the following elements, and write the script to sort the book list by author name or publication year according to the clicked button.

```
<ul id="books">
  <li data-author="Jules Verne" data-year="1872">
    Around the World in Eighty Days</li>
  <li data-author="Jules Verne" data-year="1870">
    Twenty Thousand Leagues Under the Seas</li>
  <li data-author="Jules Verne" data-year="1867">
    Journey to the Center of the Earth</li>
  <li data-author="Mark Twain" data-year="1876">
    The Adventures of Tom Sawyer</li>
  <li data-author="Liu Xu" data-year="945">
    Old Book of Tang</li>
  <li data-author="Wu Cheng'en" data-year="1592">
    Journey to the West</li>
  <li data-author="Nguyen Du" data-year="1820">
    The Tale of Kieu</li>
</ul>
<button id="by-author">Sort by author</button>
<button id="by-year">Sort by publication year</button>
```

Web APIs

Introduction

- HTML5 includes a large number of Web APIs available
  - Still under evolution
- Most important APIs:
  - DOM manipulation
  - Window and navigator
  - Navigation and history
  - Canvas, WebGL
  - Custom components & elements
  - Notification
  - AJAX, fetch, WebSockets
  - Local resource access: storage, file system, clipboard, Bluetooth, audio, camera, screen capture, geolocation,…
  - Background tasks
  - Broadcast channel
  - …

BOM: Browser Object Model

- Allows JavaScript to communicate to the browser

- The `window` object represents the browser's window
  - `window` is global `this` (or `globalThis`)
  - All global functions and variables are automatically members of `window`

- Examples:
  - `console.log(this === window);  // true`
  - `var a = 10;  // `const a`, `let a` not working`
    `console.log(window.a);  // 10`

`window` Object

- `innerWidth`, `innerHeight`: size of the content area (in px)
- `outerWidth`, `outerHeight`: size of the browser window
- `screenX`, `screenY`: position of the window relative to the screen
- `scrollX`, `scrollY`: scrolled position of the document relative to the window
- `close()`: closes the window
- `open(`*`url`*`)`: opens the given URL in a new window
- `alert(`*`msg`*`)`: displays a message box
- `prompt(`*`msg`*`)`: displays a dialog box that prompts the user for input
- `confirm(`*`msg`*`)`: displays a dialog box that asks the user to verify or accept something
- `setTimeout(`*`func, ms`*`)`, `clearTimeout(id)`: calls-once/stops-calling a function after a specified number of milliseconds
- `setInterval(`*`func, ms`*`)`, `clearInterval(`*`id`*`)`: calls-repeatedly/stops-calling a function at specified number of milliseconds

`screen` Object

- `width`, `height`: user's screen size
- `availWidth`, `availHeight`: user's screen size minus interface features like the system taskbar
- `colorDepth`: number of bits used to display one color
- `pixelDepth`: pixel depth of the screen

`navigator` Object

- `cookieEnabled`: whether cookies are enabled
- `appName`: application name of the browser
- `appCodeName`: application code name of the browser
- `appVersion`: version information about the browser
- `product`: product name of the browser engine
- `platform`: browser platform (operating system)
- `userAgent`: user-agent header sent by the browser to the server
- `language`: browser's language
- `onLine`: whether the browser is online
- `connection`: information about the system connections

`location` Object

- `href`: href (URL) of the current page
- `hostname`: domain name of the web host
- `pathname`: path and filename of the current page
- `protocol`: web protocol used (http: or https:)
- `assign(`*`url`*`)`: loads a new document
- `reload()`: reloads the current page

- Attn: the following codes are equivalent
  - `location.assign(url);`
  - `location = url;`
  - `location.href = url;`

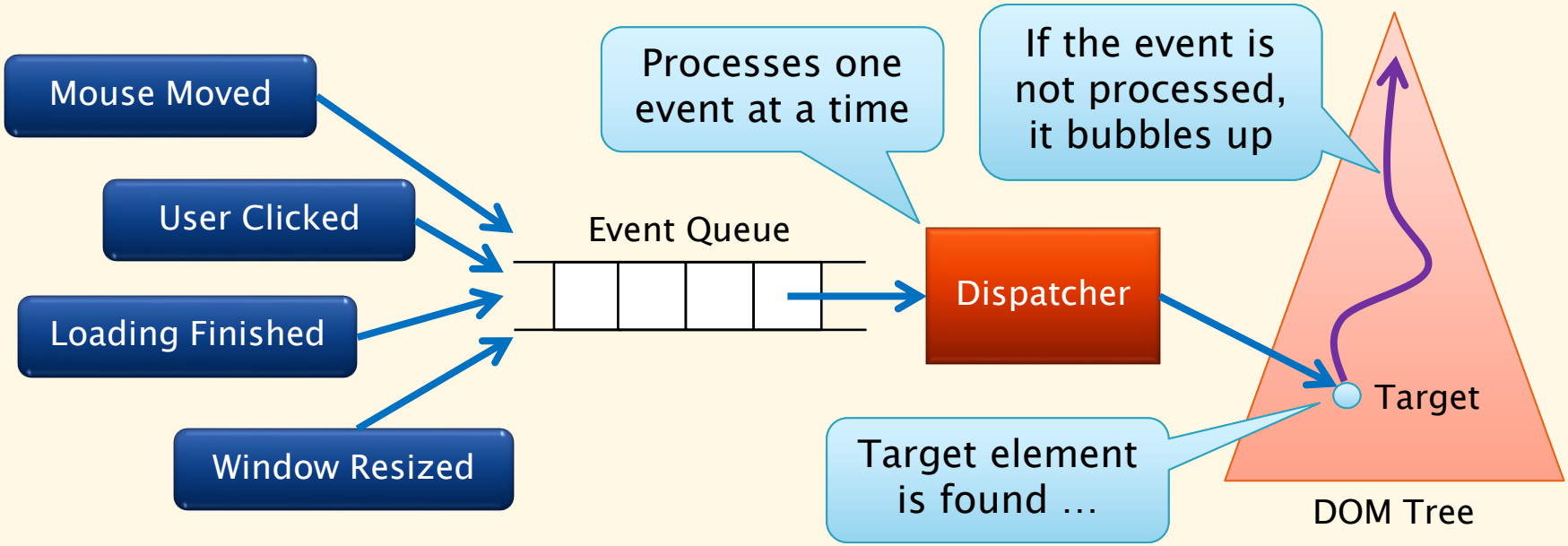
`history` Object

- `length`: number of items in the history list
- `back()`: loads the previous URL in the history list
- `forward()`: loads the next URL in the history list
- `go(n)`: loads a specific page from session history, identified by its relative position to the current page
  - *n* = 0: current page
  - *n* = -1: previous page
  - *n* = 1: next page

Events

Event Model

- Top-level scripts are executed immediately
- Other code can be attached to various events
- One event loop is processed in a single thread



Examples

- Static method:
  - `<button onclick="alert('clicked')">…</button>`
  - `<button onclick="clickHandler()">…</button>`
  - `<button onclick="clickHandler1();clickHandler2();">…</button>`
  - Handler functions may not be available when event is fired, as scripts containing them haven't been parsed!
  - Bad presentation-logic separation → painful maintenance efforts!
- Dynamic (late-binding) method:
  - `button.onclick = clickHandler;`
  - `button.onclick = function() { … };`
  - `button.addEventListener("click", clickHandler);` (multiple handler possible → Recommended method!)

Event Handler Manipulation

- `EventHandler` interface
  - Implemented by `Element`, `Document`, `Window` classes
  - `.addEventListener(`*`type`*`, `*`handler`*`)`: registers an event handler
  - `.removeEventListener(`*`type`*`, `*`handler`*`)`: removes an event listener
  - `.dispatchEvent(`*`event`*`)`: dispatches an event to the object

Event Object

- Event is represented by an object implementing `Event` interface
  - Special events may implement some other interface derived from `Event` (e.g., `MouseEvent`)
- The object carries event information
  - Common information:
    - `.target`, `.currentTarget`, `.bubbles`, `.cancelable`
  - Event specific information (e.g., mouse coordinates)
- The event propagation may be disrupted
  - `.preventDefault()`
  - `.stopPropagation()`

Example

```
<style>
#canvas {
  height: 500px;
  border: 1px solid blue;
}

.dot {
  border: 1px solid red;
  border-radius: 50%;
  display: block;
  position: absolute;
  width: 4px;
  height: 4px;
}
</style>
```

```
<div id="canvas"></div>


<script>
const c = document.getElementById('canvas');

c.onclick = function(e) {
  const a = document.createElement('div');
  a.className = 'dot';
  a.style = `left: ${e.clientX-2}px;
             top: ${e.clientY-2}px;`;
  c.appendChild(a);
}
</script>
```

Common Events

- `click`, `dblclick`: user clicks/double-clicks element
- `mouseover`, `mouseout`: user moves mouse over/away from element
- `focus`, `blur`: element gets/loses focus
- `scroll`: element's scrollbar is being scrolled
- `load`: object has been loaded
- `keypress`, `keydown`, `keyup`: user presses/releases a key

Some Element-Specific Events

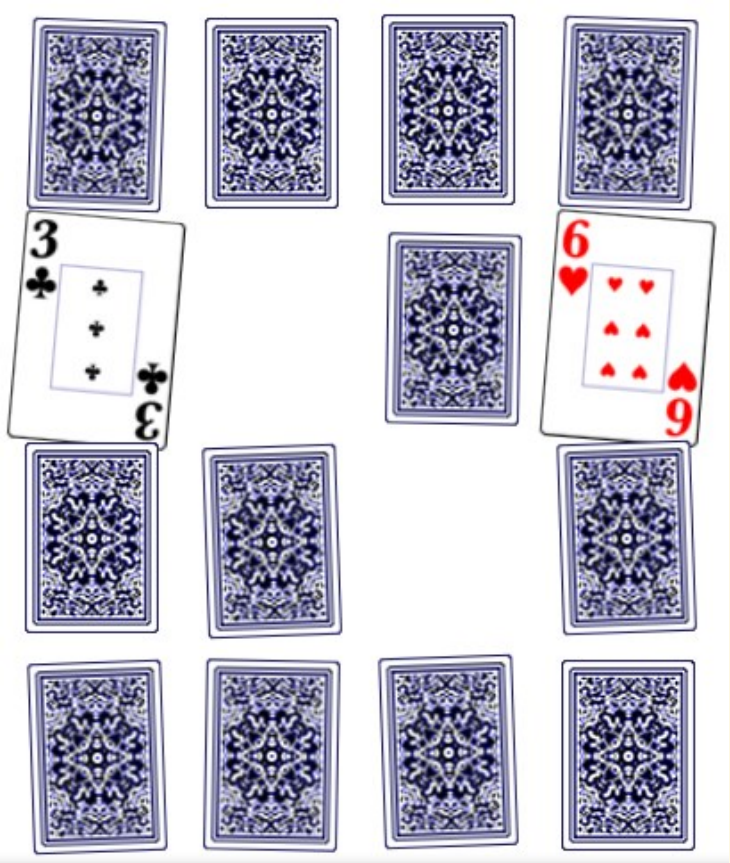
- `online`, `offline` (on `document.body`): browser has gained/lost network access
- `scroll` (on `document`): document's scrollbar is being scrolled
- `resize` (on `window`): document view is resized

Programmatically Firing Events

- Example:
  - ```
    const tg = document.getElementById('target');
    const event = new Event('click');
    tg.dispatchEvent(event);
    ```
    ```

Exercises

1. Create a page showing a big image and an initial message “Image loading…”, then remove the message once the image is finished loading. If the image loading fails, change the message to “Image loading failed!”
2. Implement a card whose face is initially down but turns up when clicking on it.
3. Make a memory card matching game.
4. Create a table that allows to sort by each column when clicking its header.

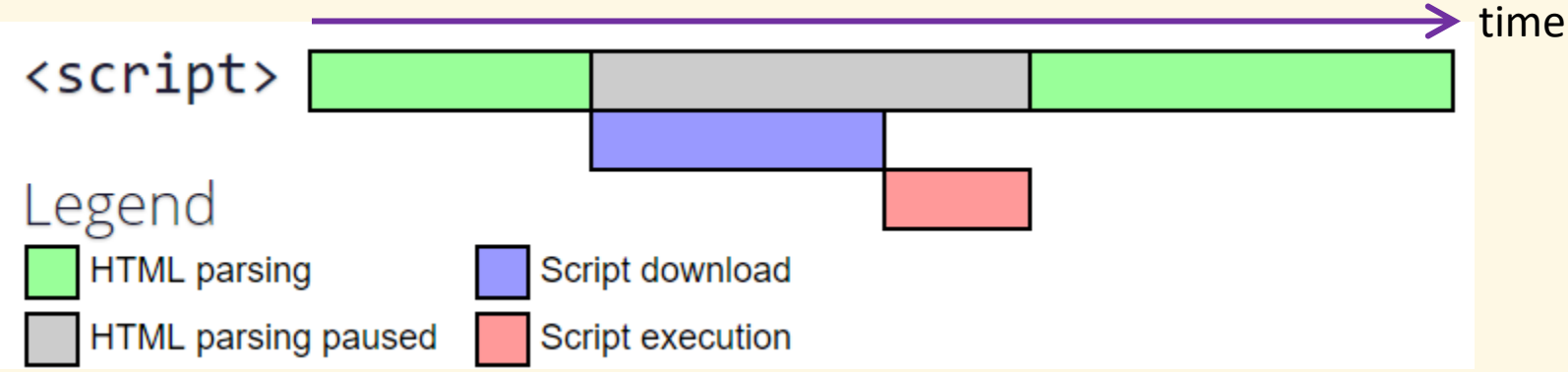


#	Student ID	Name
1	20187287	Bùi Đức Anh
20	20187347	Đỗ Thị Thùy Trang
7	20187322	Đồng Quang Huy
11	20207683	Dư Xuân Tùng Lâm
8	20168228	Hoàng Quốc Huy
14	20198138	Nguyễn Hoàng Ly
3	20207656	Nguyễn Nam Anh

More on <script> Tag

How the Browser Handles a Script

- By default (neither `async` nor `defer` is used):
  1. Fetch and parse the HTML page
  2. The parser encounters a `<script>` tag
  3. If the script is external, the browser requests the script file, the HTML parser is blocked
  4. After some time, the script is loaded and subsequently executed
  5. The parser continues parsing the rest of the HTML document
- Why? Because old developpers used to use `document.write()` to generate dynamic HTML → the parser needs to wait



Asynchronous Loading

- Use `async` attribute to make a script run asynchronously as soon as it is available
  - With: script is executed asynchronously with the rest of the page, while the page continues the parsing
  - Without: script is fetched and executed immediately, before the browser continues parsing the page
  - Attn: Only for external scripts
- Example:
  - ```
    <script src="script1.js"></script>
    <script src="script2.js" async></script>
    <script src="script3.js" async></script>
    ```
  - `script1.js` is downloaded and executed immediately, making page parser blocked until the execution finishes
  - `script2.js` and `script3.js` are downloaded immediately, then executed when available and asynchronously (i.e., in any order)
    ```

Deferred Execution

- Use `defer` attribute to make a script to be executed when the page has finished parsing
  - Attn: Only for external scripts
- Example:
  - ```
    <script src="script1.js"></script>
    <script src="script2.js" defer></script>
    <script src="script3.js" defer></script>
    ```
  - `script1.js` is downloaded and executed immediately, making page parser blocked until the execution finishes
  - `script2.js` and `script3.js` are downloaded, but executed once the page parser has finished, and in the given order
    ```

async vs defer

- Attn: When both are applied, `async` takes priority over `defer`

Where to Put Embedded Scripts?

- Consider this example:

```
<html>
<head>
  <script>
  const a = document.getElementById('msg');
  a.innerText = 'Hello';
  </script>
</head>

<body>
  <p id="msg"></p>
</body>
</html>
```

➔ The element has not been constructed when the script is executed

Solution 1

- Execute the code only when page loading has finished
  - ```
    <html>
    <head>
      <script>
      window.onload = function() {
        const a = document.getElementById('msg');
        a.innerText = 'Hello';
      }
      </script>
    </head>

    <body>

    </body>
    </html>
    ```

➔ The script parsing unnecessarily blocks the HTML parsing
    ```

Solution 2

- Bring the script to the end of the document
  - ```
    <html>
    <head>
    </head>

    <body>

      <script>
      const a = document.getElementById('msg');
      a.innerText = 'Hello';
      </script>
    </body>
    </html>
    ```
    ```

Where to Put External Scripts?

- What if the script is external?

```
<html>
<head>
</head>

<body>
  <p id="msg"></p>

  <script src="my-script.js"></script>
</body>
</html>
```

➔ The script downloading starts too late!

➔ And the script execution also starts too late!

Solution

- Bring the script to `<head>`, but use `defer`
  - ```
    <html>
    <head>
      <script src="https://xyz.com/my-script.js"
    defer></script>
    </head>

    <body>

    </body>
    </html>
    ```

➔ The script is downloaded ASAP, when the HTML is being parsed!

➔ The script execution also starts earlier!
    ```

Dynamic Script Loading

- To make a script to be loaded conditionally
- By adding a `<script>` element to the DOM:

```
function loadScript(src) {
  const s = document.createElement("script");
  s.src = src;
  s.async = false;
  document.body.append(s);
}

// big.js runs first because of async=false
loadScript("big.js");
loadScript("small.js");
```

Script Organization Strategies

- Write small scripts inline
  - No additional network connections needed
  - Put at the bottom of the document
- Make large scripts external
  - Reduce HTML document size
  - Usually cached by browsers
  - Make them asynchronous if possible, and put in `<head>`
    - If not asynchronous, put at the bottom
- Combine multiple small scripts into a single file
- Delay the loading of contextual scripts with dynamic mechanism

Security Concerns

- Use `integrity="hash-value"` attribute to ensure that the script hosted on 3rd-party servers have not been altered
  - The code is never loaded if the source has been manipulated
  - `<script src="https://code.jquery.com/jquery-3.3.1.slim.min.js" integrity="sha384-q8i/X+965DzO0rT7abK41JStQIAqVgRVzpbzo5smXKp4YfRvH+8abtTE1Pi6jizo" crossorigin="anonymous"></script>`
- Need to calculate hash value?
  - Use OpenSSL:
    - Download and install: https://www.openssl.org/
    - `openssl dgst -sha384 -binary FILENAME.js | openssl base64 -A`
  - There are online tools!

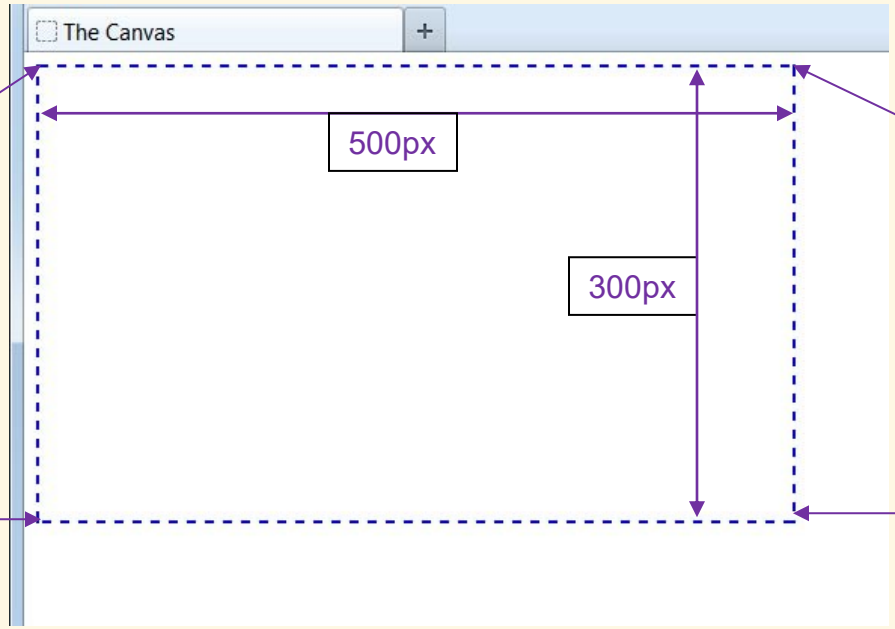
Canvas

Introduction

- Canvases enables designers and programmers to create and place graphics directly onto a web page
  - Prior to HTML5, an external plug-in, usually Adobe Flash, was required to accomplish many of the features that a canvas can now perform
  - Can be used to draw simple lines and geometric figures, but also to create highly complex and interactive games and applications
  - Absolutely requires the use of JavaScript to function
- `<canvas>` element:
  - `<canvas id="drawing" width="500" height="300"></canvas>`

Canvas Coordinates

- 2D Cartesian coordinates
- Upper limits of the *x* and *y* depend on the width and height



(0,0) is the top left corner of the canvas

(500,0) is the top right corner of the canvas

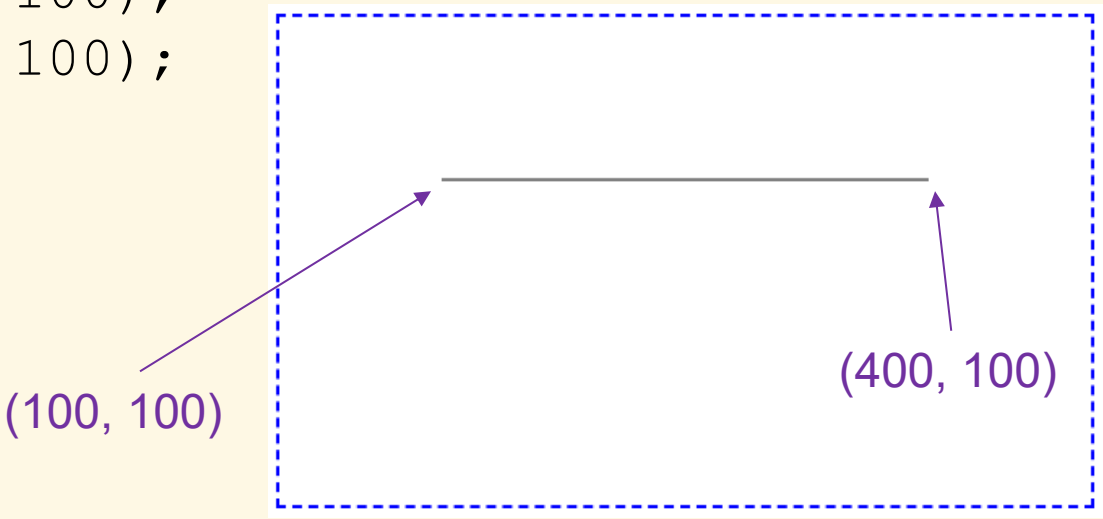
(0,300) is the bottom left corner of the canvas

(500,300) is the bottom right corner of the canvas

Lines and Paths

- Obtaining the "context" (`CanvasRenderingContext2D`):
  - ```
    const cnv = document.getElementById("drawing");
    const ctx = canvas.getContext("2d");
    ```

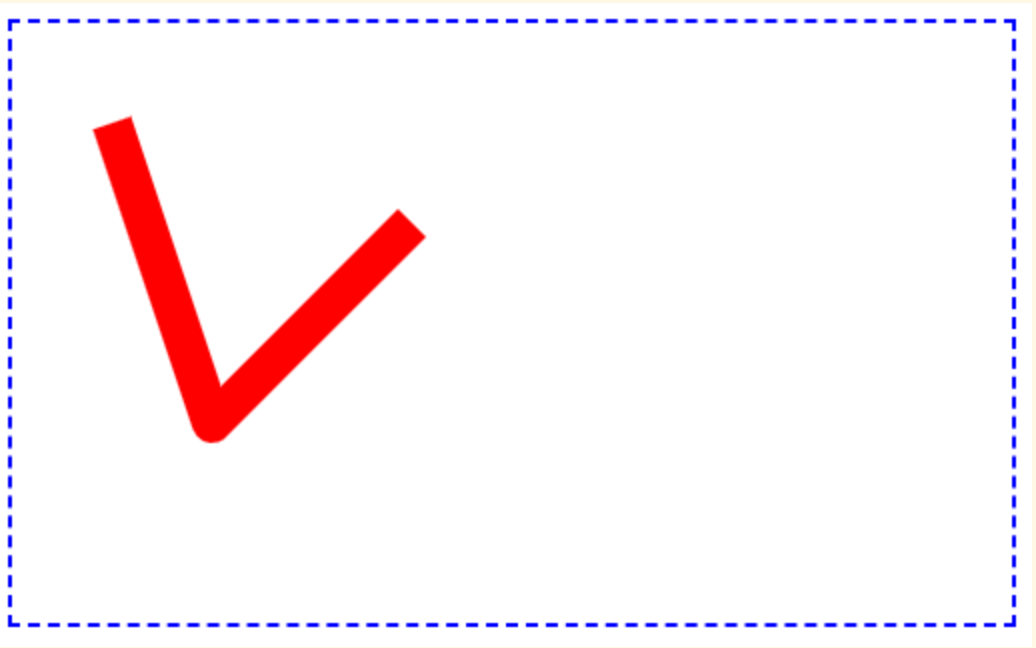
- Draw the line:
  - ```
    ctx.beginPath();
    ctx.moveTo(100, 100);
    ctx.lineTo(400, 100);
    ctx.stroke();
    ```



Line Options

```
ctx.lineWidth = 20;
ctx.strokeStyle = "red";
ctx.lineCap = "square";
ctx.lineJoin = "round";

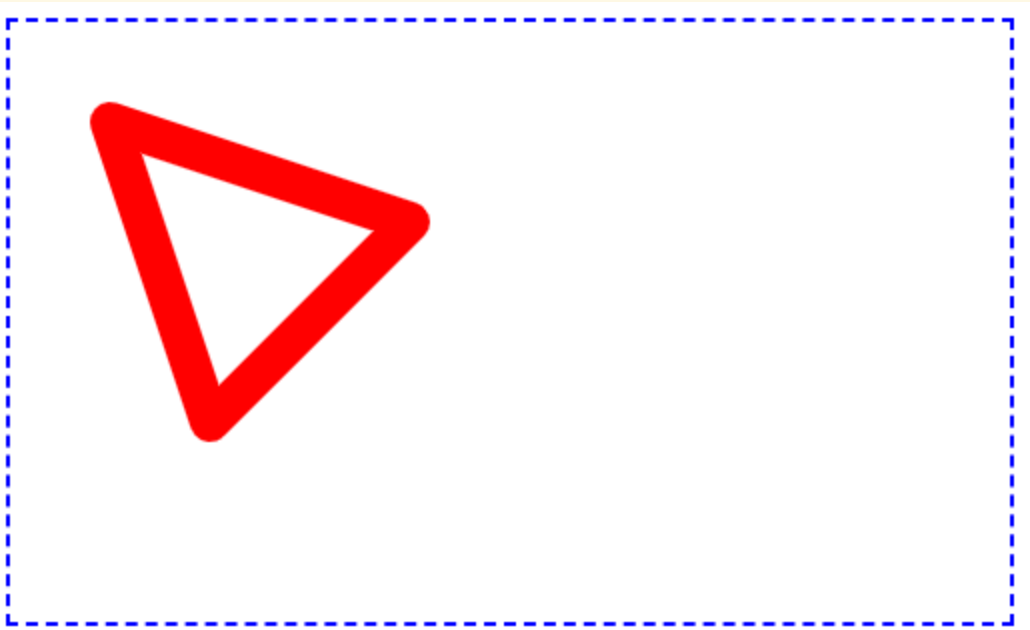
ctx.beginPath();
ctx.moveTo(50, 50);
ctx.lineTo(100, 200);
ctx.lineTo(200, 100);
ctx.stroke();
```



Closed Paths

```
ctx.lineWidth = 20;
ctx.strokeStyle = "red";
ctx.lineCap = "round";
ctx.lineJoin = "round";

ctx.beginPath();
ctx.moveTo(50, 50);
ctx.lineTo(100, 200);
ctx.lineTo(200, 100);
ctx.closePath();
ctx.stroke();
```



More Line Options

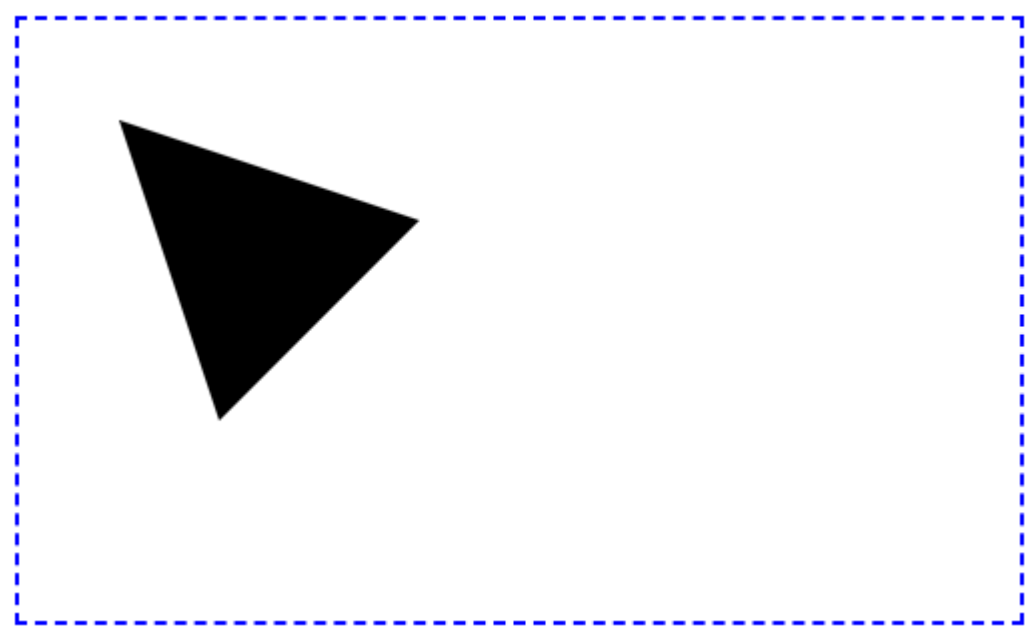
- `lineCap = "butt"|"round"|"square"`

- `lineJoin = "round"|"bevel"|"miter"`

- `setLineDash(`*`pattern`*`)`
  - Ex: `setLineDash([3, 8, 10, 2])`

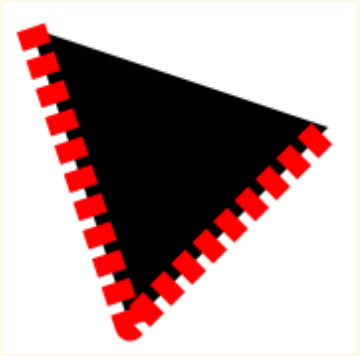
Filled Shapes

```
ctx.beginPath();
ctx.moveTo(50, 50);
ctx.lineTo(100, 200);
ctx.lineTo(200, 100);
ctx.fill();
```

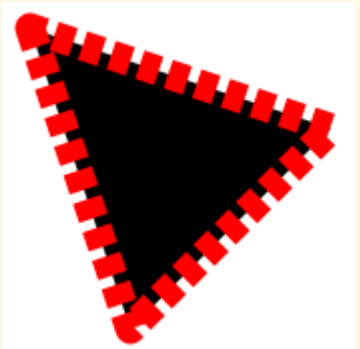


`fill()` and `stroke()` together

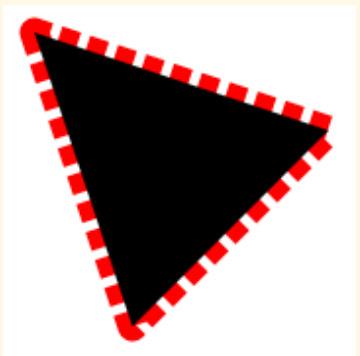
```
...
ctx.fill();
ctx.stroke();
```



```
...
ctx.closePath();
ctx.fill();
ctx.stroke();
```

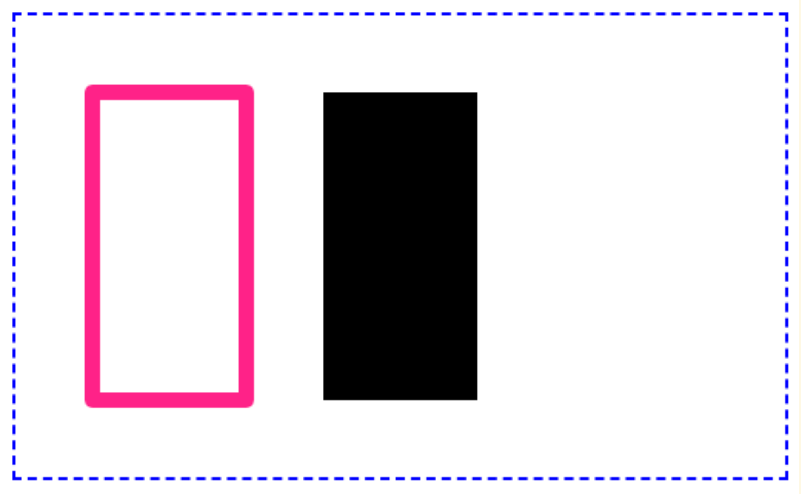


```
...
ctx.closePath();
ctx.stroke();
ctx.fill();
```



Rectangles

- `strokeRect(`*`left, top, width, height`*`)`
- `fillRect(`*`left, top, width, height`*`)`
- Example:
  - ```
    ctx.lineWidth = 10;
    ctx.strokeStyle = "#f28";
    ctx.lineJoin = "round";
    ctx.strokeRect(50, 50, 100,
    200);
    ctx.fillRect(50, 50, 100,
    200);
    ```


    ```

More Paths

- `rect(x, y, width, height)`
- `ellipse(x, y, rx, ry, rotation, startAng, endAng)`
- `arc(x, y, r, startAng, endAng)`
- `arcTo(x1, y1, x2, y2, r)`
- `quadraticCurveTo(cpx, cpy, x, y)`
- `bezierCurveTo(cp1x, cp1y, cp2x, cp2y, x, y)`

`arcTo`

`quadraticCurveTo`

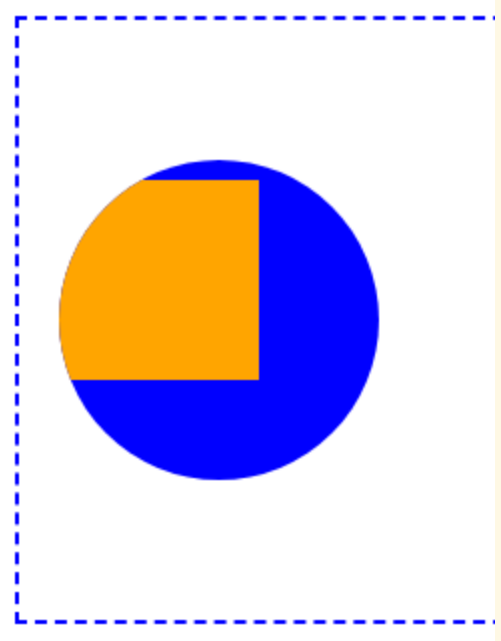
`bezierCurveTo`

Fill Styles

- `fillStyle = color|gradient|pattern;`
- Examples:
  - `ctx.fillStyle = "yellow";`
  - `ctx.fillStyle = "#256f7a";`
  - `ctx.fillStyle = "rgb(127, 0, 255)";`
  - ```
    const img = new Image();
    img.src = "pattern.png";
    img.onload = function() {
      const pat = ctx.createPattern(img, "repeat");
      ctx.fillStyle = pat;
    };
    ```
  - ```
    const grad = ctx.createLinearGradient(20, 0, 220, 0);
    grad.addColorStop(0, 'green');
    grad.addColorStop(.5, 'cyan');
    grad.addColorStop(1, 'green');
    ctx.fillStyle = grad;
    ```

Clipping

- `clip([path])`: turns the current or given path into the current clipping region

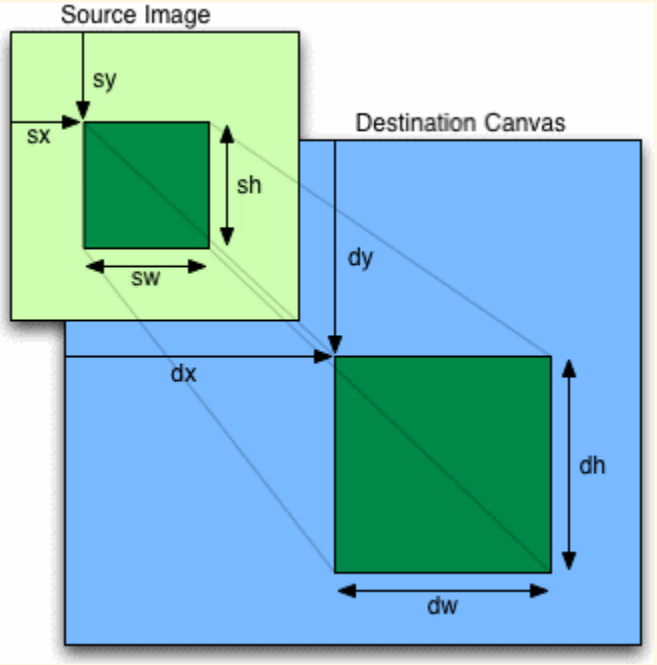


- Example:

```
ctx.beginPath();
ctx.arc(100, 75, 50, 0, Math.PI * 2);
ctx.clip();

ctx.fillStyle = 'blue';
ctx.fillRect(0, 0, canvas.width, canvas.height);
ctx.fillStyle = 'orange';
ctx.fillRect(0, 0, 100, 100);
```

Images



- Syntax:
  - `drawImage(img, dx, dy)`
  - `drawImage(img, dx, dy, dw, dh)`
  - `drawImage(img, sx, sy, sw, sh, dx, dy, dw, dh)`
- Example:

```
let img = new Image();
img.src = "picture.jpg";
// or: img.src = "…";

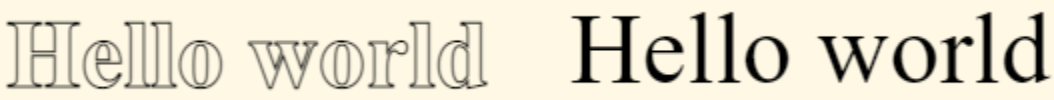
img.onload = function() {
  ctx.drawImage(img, 100, 50);
};
```

- Question: Is it neccessary to set `img.src` before `img.onload`?

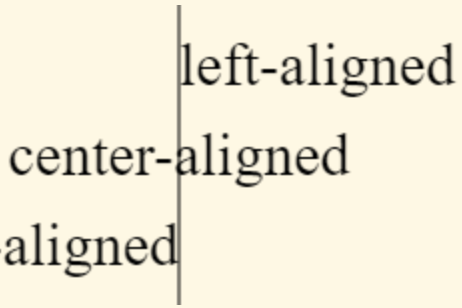
Text

- Syntax:
  - `strokeText(text, x, y, [maxWidth])`
  - `fillText (text, x, y, [maxWidth])`
- Options:
  - `font =` CSS `font` value
  - `textAlign = "left"|"right"|"center"|"start"|"end"`
  - `textBaseline = "alphabetic"|"top"|"middle"|"bottom"`
- Examples:

```
ctx.font = 'bold 48px serif';
ctx.strokeText('Hello world', 50, 100);
ctx.fillText('Hello world', 250, 90, 140);
```

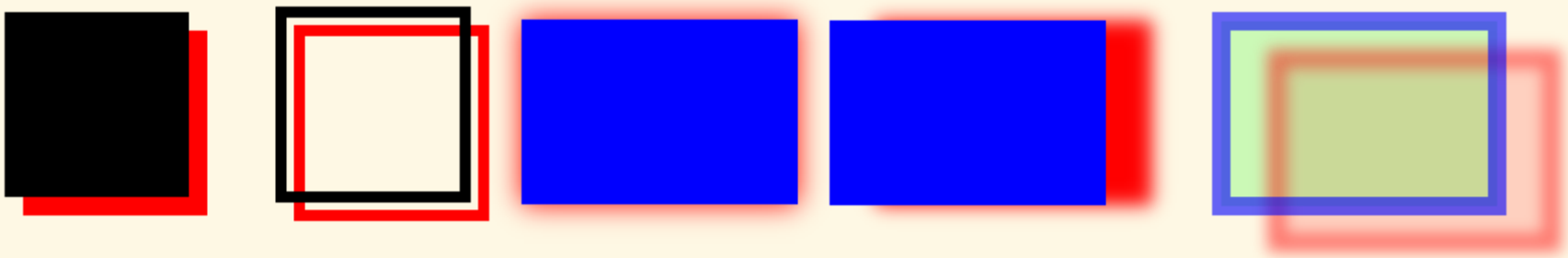


`strokeText` `fillText`



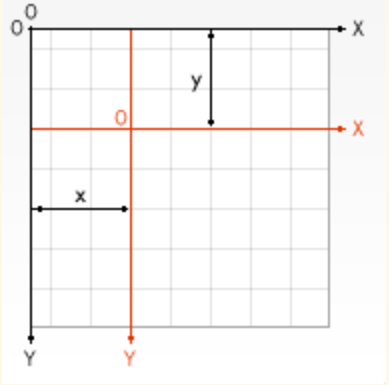
Shadows

- Use the following properties:
  - `shadowColor`
  - `shadowBlur`
  - `shadowOffsetX`
  - `shadowOffsetY`

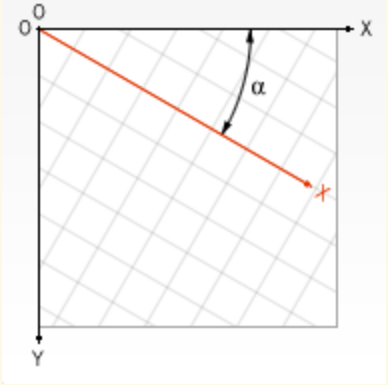


Transformations

- Translation:
  - `translate(x, y)`
- Rotation:
  - `rotate(ang)`
- Scaling:
  - `scale(sx, sy)`



`translate`



`rotate`

- By matrix (general case):

$$\begin{bmatrix} m_{11} & m_{12} & dx \\ m_{21} & m_{22} & dy \\ 0 & 0 & 1 \end{bmatrix}$$

  - `transform(m11, m12, m21, m22, dx, dy)`
- Save and restore transformation (and style) state:
  - `save()`
  - `restore()`

Animations

- Principle: When the content changes, notify the browser that the canvas needs to be updated before the next repaint
  → the browser will invoke a given draw function

- Steps to redraw a frame:
  1. Clear the canvas: using `clearRect()`
  2. Save the canvas state: using `save()`
  3. Draw shapes: using `window.requestAnimationFrame()`
     Attn: `requestAnimationFrame` is valid until the next repaint
  4. Restore the canvas state: using `restore()`

- Question: Why not call the draw function directly in Step 3?

Example: Rotating Dot

```
const w = canvas.width;
const h = canvas.height;

// initial direction
let dir = 0;

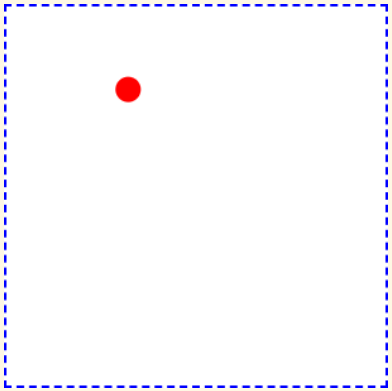
// update direction every 50ms
setInterval(function() {
  dir += 0.02;

  // notify the browser to redraw
  window.requestAnimationFrame(draw);
}, 50);

// first frame
draw();
```

```
function draw() {
  ctx.clearRect(0, 0, w, h);
  ctx.save();
  ctx.translate(w/2, h/2);
  ctx.rotate(dir);
  ctx.translate(100, 0);

  ctx.fillStyle = "red";
  ctx.beginPath();
  ctx.ellipse(0, 0, 10, 10, 0,
              0, Math.PI*2);
  ctx.fill();
  ctx.restore();
}
```



Exercises

1. Make the animation in previous example paused, resumed on mouse clicks.
2. Create an analog clock as in the following figure.

SVG

Introduction

- SVG: Scalable Vector Graphics
  - XML-based markup language
  - 2D vector graphics
- Advantages of SVG over bitmap graphics (canvas, JPEG, PNG,…):
  - Can be rendered at any size without loss of quality
  - Can be searched, indexed
  - Easy to edit with any text editor
  - Easier to manipulate graphic objects
- Drawbacks
  - Slower rendering if complex
  - Not very suited for gaming applications

Examples

- Embedded SVG:

```
<svg width="300" height="200" xmlns="http://www.w3.org/2000/svg">
  <rect width="100%" height="100%" fill="red" />
  <circle cx="150" cy="100" r="80" fill="green" />
  <text x="150" y="125" font-size="60"
     text-anchor="middle" fill="white">SVG</text>
</svg>
```

- External SVG:

```
<object data="image.svg" type="image/svg+xml"></object>
```

Shapes

```
<svg width="100" height="250" xmlns="http://www.w3.org/2000/svg">
  <rect x="10" y="10" width="30" height="30" stroke="black"
      fill="transparent" stroke-width="5" />
  <rect x="60" y="10" rx="10" ry="10" width="30" height="30"
      stroke="black" fill="transparent" stroke-width="5" />

  <circle cx="25" cy="75" r="20" stroke="red"
      fill="transparent" stroke-width="5" />
  <ellipse cx="75" cy="75" rx="20" ry="5" stroke="red"
      fill="transparent" stroke-width="5" />

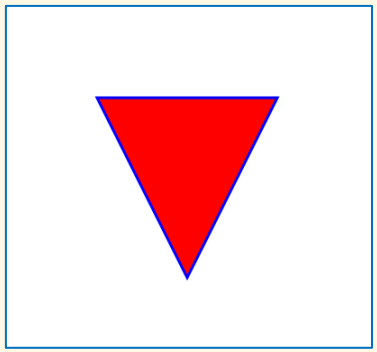
  <line x1="10" x2="50" y1="110" y2="150" stroke="orange"
      stroke-width="5" />
  <polyline points="60 110 65 120 70 115 75 130 80 125 85 140 90 135 95
150 100 145" stroke="orange" fill="transparent" stroke-width="5" />

  <polygon points="50 160 55 180 70 180 60 190 65 205 50 195 35 205 40
190 30 180 45 180" stroke="green" fill="transparent" stroke-width="5" />
</svg>
```

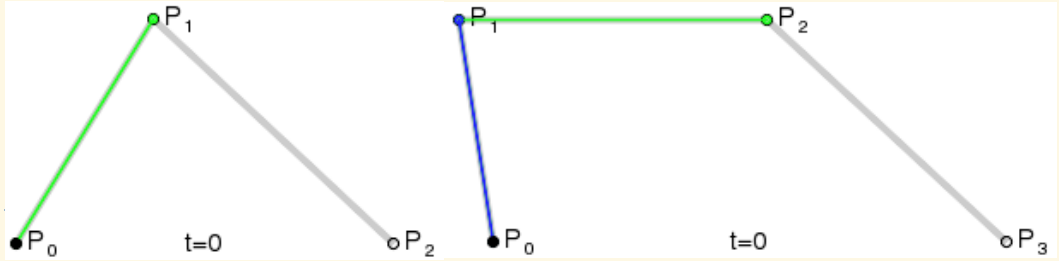
Paths

- `<path>`: generic element to define a complex shape with multiple lines, curves, arcs,… by a series of commands

- Line commands:
  - `M` *x y* or `m` *dx dy*: move to (*x*, *y*) or relatively (*dx*, *dy*)
  - `L` *x y* or `l` *dx dy*: line to
  - `H` *x* or `h` *dx*: horizontal line
  - `V` *y* or `v` *dy*: vertical line
  - `Z` or `z`: close path
- Example:
  - `<path d="M 100 100 L 300 100 L 200 300 z"`
    `fill="red" stroke="blue" stroke-width="3" />`

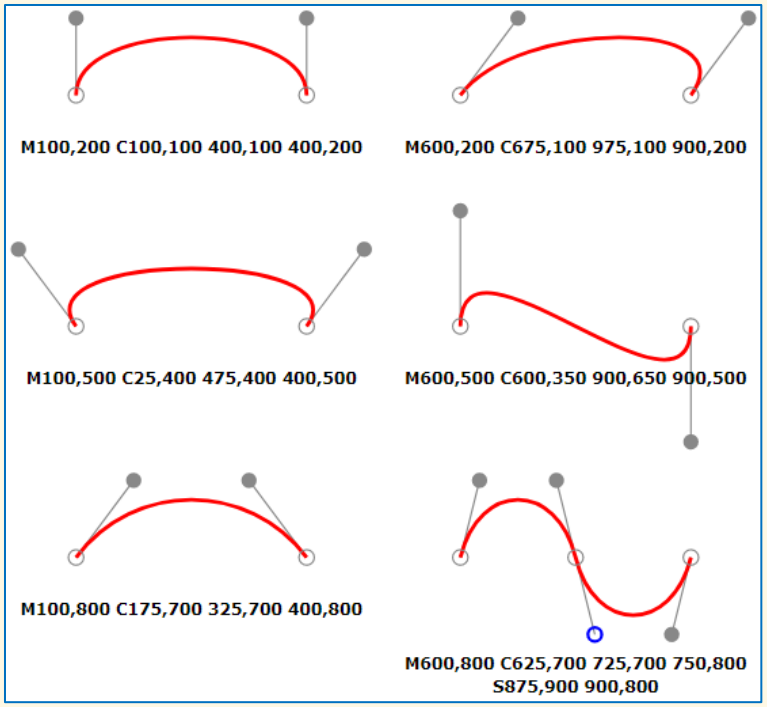


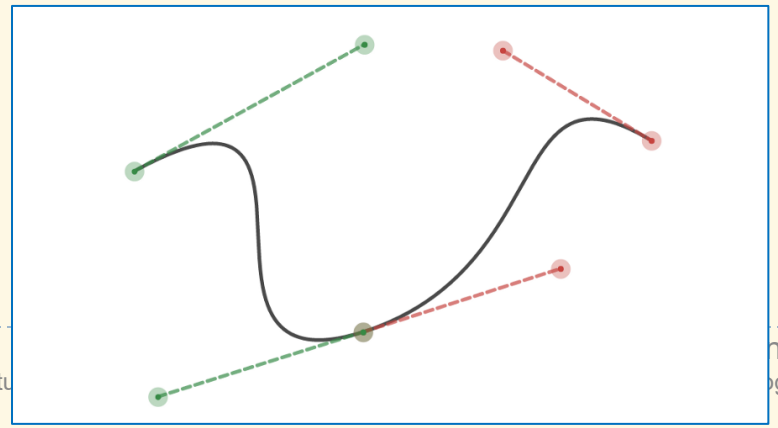
Paths: Bézier Curves



- Quadratic curve:
  - `Q x1 y1, x y`
  - `q dx1 dy1, dx dy`
- Cubic curve:
  - `C x1 y1, x2 y2, x y`
  - `c dx1 dy1, dx2 dy2, dx dy`

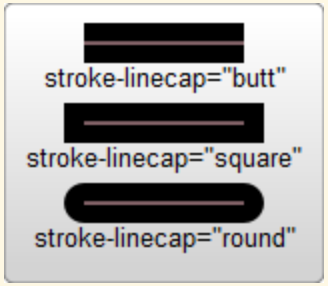
  where:
  - (*x1*, *y1*), (*x2*, *y2*): control points
  - (*x*, *y*): end point
- Curve stringing:
  - `T x y`
  - `t dx dy`
  - `S x2 y2, x y`
  - `s dx2 dy2, dx dy`





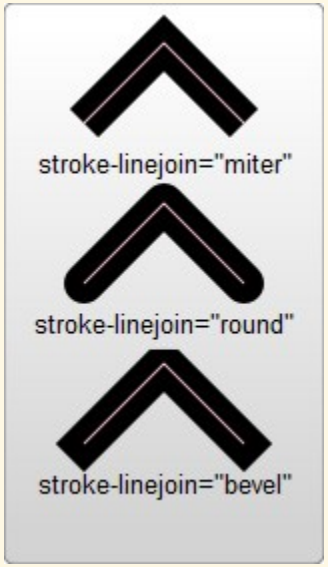
Fill and Stroke Styles

- Shape attributes:
  - `stroke`, `stroke-width`, `stroke-opacity`, `stroke-linecap`, `stroke-linejoin`, `stroke-dasharray`
  - `fill`, `fill-opacity`

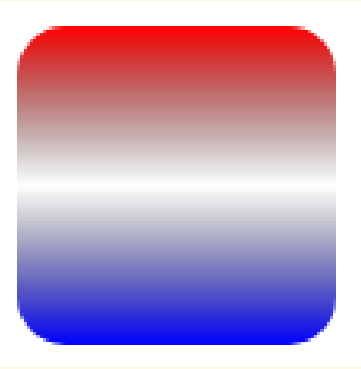


- Also available in CSS:

```
<svg width="200" height="200" ...>
  <defs>
    <style type="text/css"><![CDATA[
      #r1 {
        stroke: black;
        fill: red;
      }
    ]]></style>
  </defs>
  <rect id="r1" x="10" height="180" y="10" width="180" />
</svg>
```



Gradients: Using Attributes



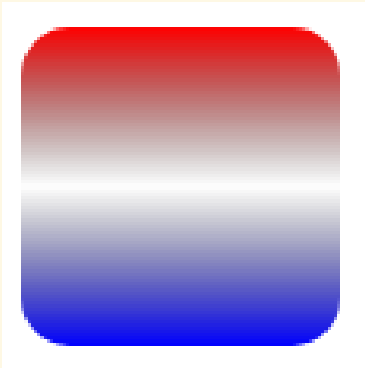
```
<svg width="120" height="240" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="grad" x1="0" x2="0" y1="0" y2="1">
      <stop offset="0%" stop-color="red"/>
      <stop offset="50%" stop-color="black" stop-opacity="0"/>
      <stop offset="100%" stop-color="blue"/>
    </linearGradient>
  </defs>

  <rect x="10" y="10" rx="15" ry="15" width="100" height="100"
        fill="url(#grad)"/>
</svg>
```

Gradients: Using CSS

```
<svg width="120" height="240" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="grad">
      <stop class="stop1" offset="0%"/>
      <stop class="stop2" offset="50%"/>
      <stop class="stop3" offset="100%"/>
    </linearGradient>
    <style type="text/css"><![CDATA[
      #r1 { fill: url(#grad); }
      .stop1 { stop-color: red; }
      .stop2 { stop-color: black; stop-opacity: 0; }
      .stop3 { stop-color: blue; }
    ]]></style>
  </defs>

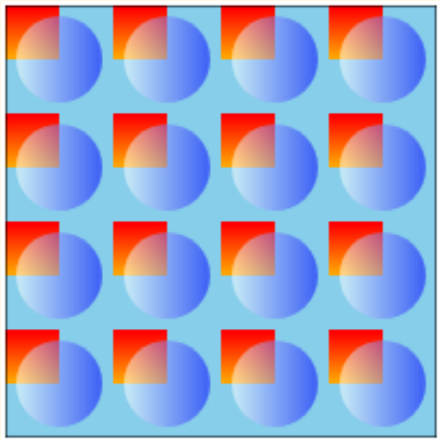
  <rect id="r1" x="10" y="10" rx="15" ry="15" width="100" height="100" />
</svg>
```



Fill Patterns

```
<svg width="200" height="200"
     xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="grad1">
      <stop offset="5%" stop-color="white"/>
      <stop offset="95%" stop-color="blue"/>
    </linearGradient>
    <linearGradient id="grad2" x1="0" x2="0" y1="0" y2="1">
      <stop offset="5%" stop-color="red"/>
      <stop offset="95%" stop-color="orange"/>
    </linearGradient>
    <pattern id="pat" x="0" y="0" width=".25" height=".25">
      <rect x="0" y="0" width="50" height="50" fill="skyblue"/>
      <rect x="0" y="0" width="25" height="25" fill="url(#grad2)"/>
      <circle cx="25" cy="25" r="20" fill="url(#grad1)" fill-opacity="0.5"/>
    </pattern>
  </defs>

  <rect fill="url(#pat)" stroke="black" width="200" height="200" />
</svg>
```



Texts

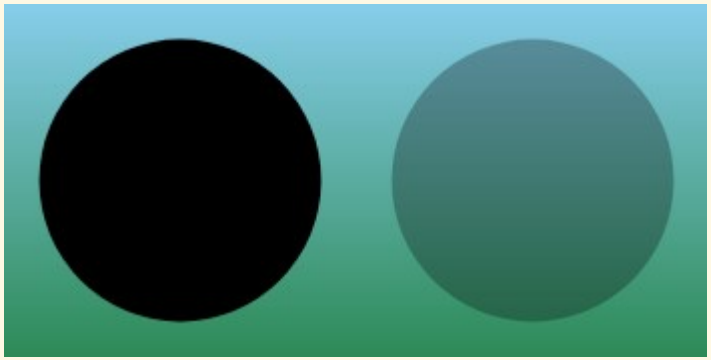
- Shape element
  - `<text x="10" y="10">Hello World!</text>`
  - ```
    <text>This is
      <tspan font-weight="bold" fill="red">
        big and red
      </tspan>
    </text>
    ```
  - ```
    <path id="p" fill="none"
       d="M 20,20 C 80,60 100,40 120,20" />
    <text>
      <textPath href="#p">A curved text</textPath>
    </text>
    ```
- Attributes/CSS properties:
  - `font-family`, `font-style`, `font-weight`, `font-variant`, `font-stretch`, `font-size`, `font-size-adjust`, `kerning`, `letter-spacing`, `word-spacing`, `text-decoration`, `text-anchor`, `dominant-baseline`

Images

- Raster image:
  - `<image x="90" y="65" width="128" height="146" href="picture.png" />`
  - `<image x="90" y="65" width="128" height="146" href="..." />`
- SVG in SVG:
  - `<image x="10" y="20" width="80" height="80" href="external.svg" />`
  - ```
    <svg ...>
      <svg ...>
      ...
      </svg>
    </svg>
    ```
    ```

Opacity

- Use `opacity` attribute/CSS property to make a whole shape opaque
- Attn: For stroke and fill opacity, use `stroke-opacity` and `fill-opacity` instead; for non-uniform opacity, use `mask`

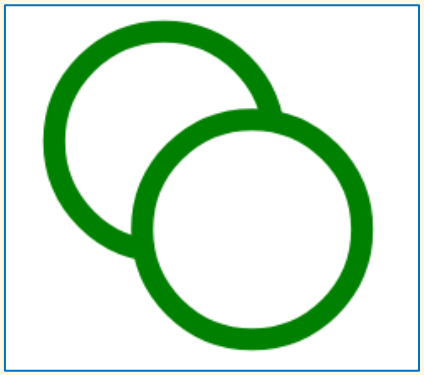


- Example:

```
<defs>
  <linearGradient id="g1" x1="0%" y1="0%" x2="0" y2="100%">
    <stop offset="0%" stop-color="skyblue" />
    <stop offset="100%" stop-color="seagreen" />
  </linearGradient>
</defs>
<rect x="0" y="0" width="100%" height="100%" fill="url(#g1)" />
<circle cx="50" cy="50" r="40" fill="black" />
<circle cx="150" cy="50" r="40" fill="black" opacity="0.3" />
```

Grouping

- Use <g> to group elements together
  - Children inherit presentation attributes
  - Apply transformation all together
  - Repeat a grouped shape multiple times with <use>

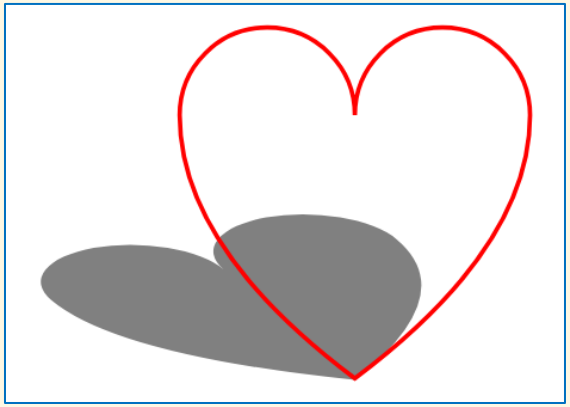


- Example:

```
<svg viewBox="0 0 100 100"
     xmlns="http://www.w3.org/2000/svg">
  <g fill="white" stroke="green" stroke-width="5">
    <circle cx="40" cy="40" r="25" />
    <circle cx="60" cy="60" r="25" />
  </g>
</svg>
```

Transformations

- Use `transform` attributes/CSS properties
  - `translate(dx, dy),rotate(ang),scale(sx, sy),skewX(a),skewY(a)`
  - `matrix(m11, m12, m21, m22, dx, dy)`



- Example:

```
<svg viewBox="-40 0 150 100"
     xmlns="http://www.w3.org/2000/svg">
  <g fill="grey"
     transform="rotate(-10 50 100) translate(-36 45.5)
                skewX(40) scale(1 0.5)">
    <path id="heart" d="M 10,30 A 20,20 0,0,1 50,30
       A 20,20 0,0,1 90,30 Q 90,60 50,90
       Q 10,60 10,30 z" />
  </g>

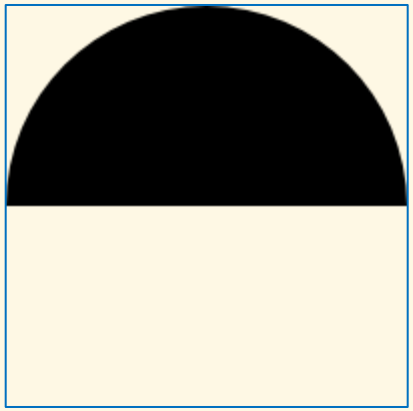
  <use href="#heart" fill="none" stroke="red"/>
</svg>
```

Clipping

- Removing parts of elements defined by other parts
- Example:

```
<svg xmlns="http://www.w3.org/2000/svg">
  <defs>
    <clipPath id="clip1">
      <rect x="0" y="0" width="200" height="100" />
    </clipPath>
  </defs>

  <circle cx="100" cy="100" r="100"
      clip-path="url(#clip1)" />
</svg>
```

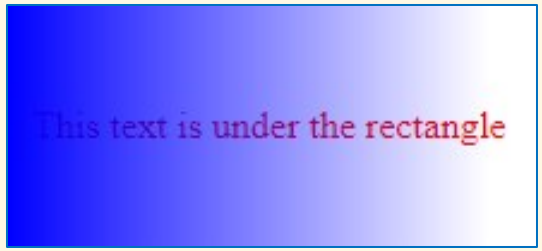


Masking

- Making parts of elements defined by other parts opaque or transparent
- Example:

```
<svg xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="grad1">
      <stop offset="0" stop-color="white" />
      <stop offset="1" stop-color="black" />
    </linearGradient>
    <mask id="mask1" x="0" y="0" width="200" height="100">
      <rect x="0" y="0" width="200" height="100" fill="url(#grad1)" />
    </mask>
  </defs>

  <text x="10" y="55" fill="red">
      This text is under the rectangle
  </text>
  <rect x="0" y="0" width="200" height="100"
      fill="blue" mask="url(#mask1)" />
</svg>
```



Filters

- A filter = ordered collection of primitives
  - Filter primitives: `feGaussianBlur`, `feDropShadow`, `feMorphology`, `feDisplacementMap`, `feBlend`, `feColorMatrix`, `feConvolveMatrix`, `feComponentTransfer`, `feSpecularLighting`, `feDiffuseLighting`, `feFlood`, `feTurbulence`, `feImage`, `feTile`, `feOffset`, `feComposite`, `feMerge`
- Defining and applying a filter:

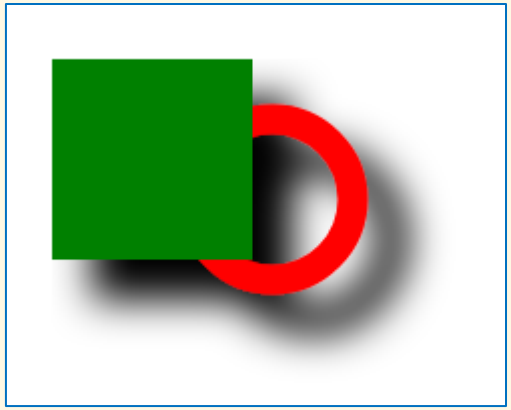
```
<svg>
  <defs>
    <filter id="filter1">
        ...
    </filter>
  </defs>

  <g filter="url(demoFilter)">
    ...
  </g>
</svg>
```

Example

```
<svg width="300" height="300">
  <defs>
    <filter id="f1" x="0" y="0" width="200%" height="200%">
      <feOffset result="offOut" in="SourceAlpha" dx="20" dy="20" />
      <feGaussianBlur result="blurOut" in="offOut" stdDeviation="10" />
      <feBlend in="SourceGraphic" in2="blurOut" mode="normal" />
    </filter>
  </defs>

  <g filter="url(#f1)">
    <circle cx="160" cy="120" r="40"
       stroke="red" stroke-width="15" fill="none" />
    <rect x="50" y="50" width="100" height="100"
       stroke="none" fill="green" />
  </g>
</svg>
```



Animation with SMIL

- SMIL: Synchronized Multimedia Integration Language
- Nest one or more of the following elements inside shapes: `animate`, `animateTransform`, `animateColor`, `animateMotion`
- Examples:
  - ```
    <circle cx="30" cy="30" r="25" stroke="none" fill="red">
      <animate attributeName="cx" attributeType="XML"
        from="30" to="470" begin="0s" dur="5s"
        fill="remove" repeatCount="indefinite" />
    </circle>
    ```
  - ```
    <rect x="20" y="20" width="100" height="40" stroke="red">
      <animateTransform attributeName="transform" type="rotate"
        from="0 100 100" to="360 100 100"
        begin="0s" dur="10s" repeatCount="indefinite" />
    </rect>
    ```
  - ```
    <rect x="0" y="0" width="30" height="15" stroke="red">
      <animateMotion path="M10,50 q60,50 100,0" rotate="auto"
        begin="0s" dur="10s" repeatCount="indefinite" />
    </rect>
    ```
    ```

SVG Scripting

- SVG elements are in the DOM → possible to manipulate them using JavaScript
  - Add/remove/modify elements dynamically
  - Animate shapes with more complex scenarios
  - Listen for events (i.e., mouse move, click) on the shapes
- Example:

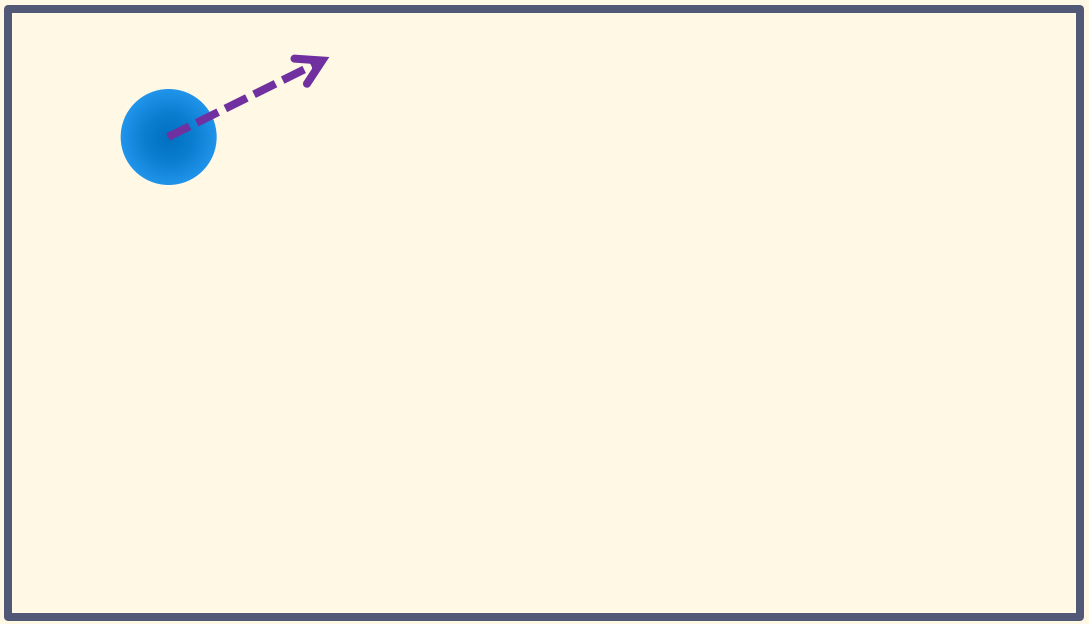
```
<svg width="500" height="100">
  <rect id="r1" x="10" y="10" width="50" height="80"/>
</svg>

<script>
  const r = document.getElementById("r1");
  let a = 0;

  setInterval(() => {
    r.setAttribute("transform", `rotate(${a})`);
    a += 2;
  }, 50);
</script>
```

Exercise

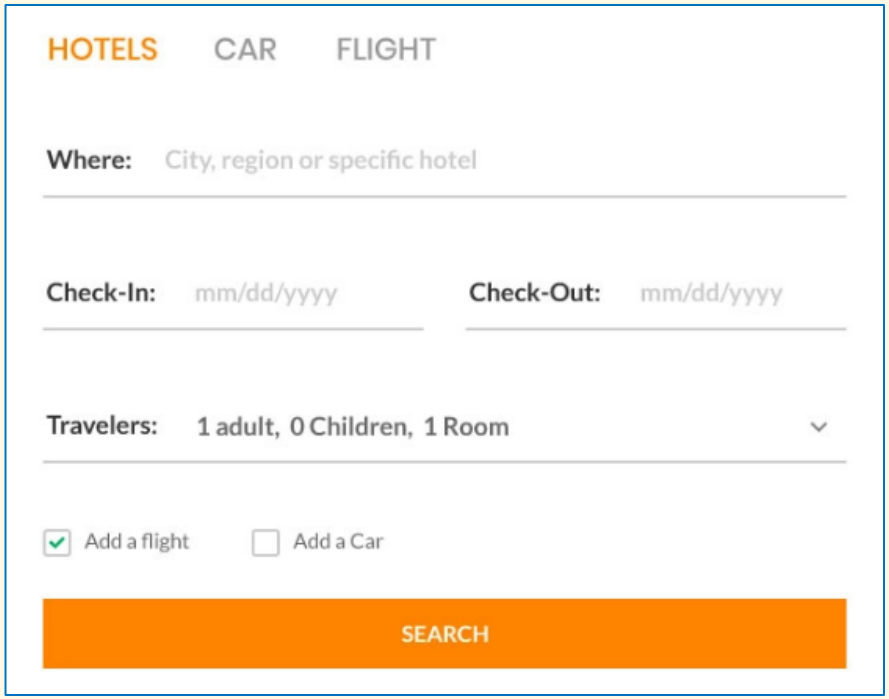
1. Create a ball bouncing within a rectangle using JavaScript. Make the ball paused, resumed on mouse clicks.
2. Create an analog clock as in the figure.



Web Forms

What are Web Forms?

- Forms allow users to enter data, which
  - Makes web apps more useful
  - Is sent to web server for processing or storage (studied in Part 4)



- A form consists of form controls (buttons, checkboxes, textboxes, dropdown boxes,...)
  - Note that buttons, checkboxes,... are not necessarily parts of a form
- Form design and implementation are parts of most important tasks in web development

Form Design

- Before writing any HTML, CSS or JavaScript, let's think about:
  - What information do you need from the user?
  - What's the best manner to ask for user input?
    - Which control to use
    - In which order
    - …
  - How to best present the control?

➔ Design a form structure first!

First Example


```
<form>
  <h2>Registration</h2>

  <p>
    <label for="name">Name:</label>
    <input type="text" id="name" name="user_name">
  </p>

  <p>
    <label for="mail">E-mail:</label>
    <input type="email" id="mail" name="user_email">
  </p>

  <button>Register</button>
</form>
```

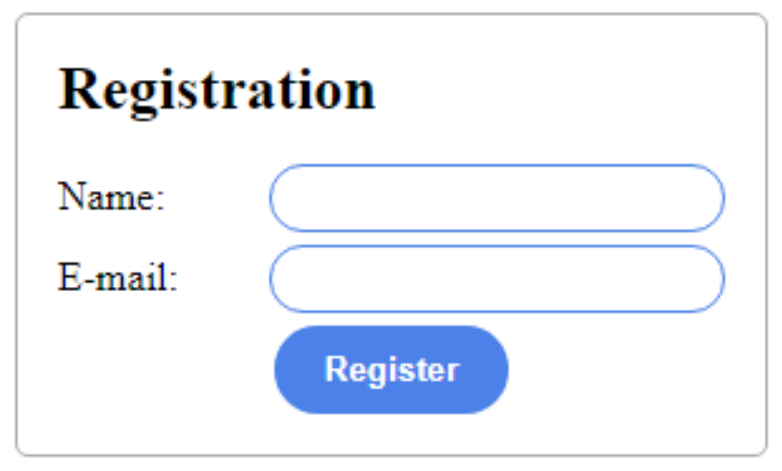
Adding Styles

```
form {
  border: 1px solid #aaa;
  border-radius: 5px;
  width: fit-content;
  max-width: 20rem;
  padding: 1rem;
  margin: auto;
}

h2 {
  margin: 0 0 1rem 0;
}

.form-field {
  margin: 5px 0;
}

label {
  width: 5rem;
  display: inline-block;
}
```




```
input {
  border: 1px solid #4b81e8;
  border-radius: 10rem;
  padding: 5px;
}

button {
  padding: 10px 20px;
  color: white;
  background: #4b81e8;
  border: transparent;
  border-radius: 10rem;
  font-weight: bold;
  display: block;
  margin: auto;
}
```

<label>

- The formal way to define a label for a form control
  - Labels are clickable too

- Examples:
  - ```
    <label for="answ">Answer:</label>
    <input type="text" id="answ" name="answer">
    ```

  - ```
    <label>Answer:
      <input type="text" name="answer">
    </label>
    ```

<input>

- Represents a form control that accepts data from the user
- There are many types of input data and control widgets: `text`, `number`, `range`, `checkbox`, `radio`, `password`, `color`, `date`, `time`, `email`, `url`, `tel`, `file`, `hidden`, `reset`, `button`, `submit`
  - Type specified by `type` attribute
- Common useful attributes:
  - `disabled` (boolean): whether the control is disabled
  - `readonly` (boolean): the value is not editable
  - `required` (boolean): the user is required to input a value
  - `value` (string): initial or input value of the control
  - `autofocus` (boolean): the control gets focus when the page is loaded
  - `placeholder` (string): text shown in the control when it has no value

Example

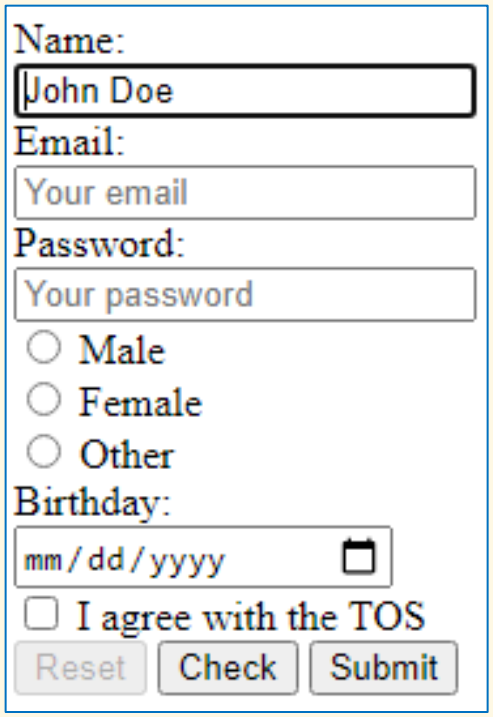
```
<form>
  <label for="name">Name:</label><br/>
  <input type="text" id="name" name="name" value="John Doe" autofocus /><br/>
  <label for="email">Email:</label><br/>
  <input type="email" id="email" name="email" placeholder="Your email" /><br/>
  <label for="pwd">Password:</label><br/>
  <input type="password" id="pwd" name="pwd"
      placeholder="Your password" readonly /><br/>

  <input type="radio" id="male" name="gender" value="male" />
  <label for="male">Male</label><br/>
  <input type="radio" id="female" name="gender" value="female" />
  <label for="female">Female</label><br/>
  <input type="radio" id="other" name="gender" value="other" />
  <label for="other">Other</label><br/>

  <label for="birthday">Birthday:</label><br/>
  <input type="date" id="birthday" name="birthday" /><br/>

  <input type="checkbox" id="agree" name="agree" value="Bike" />
  <label for="agree">I agree with the TOS</label><br/>

  <input type="reset" value="Reset" disabled />
  <input type="button" value="Check" />
  <input type="submit" value="Submit" />
</form>
```



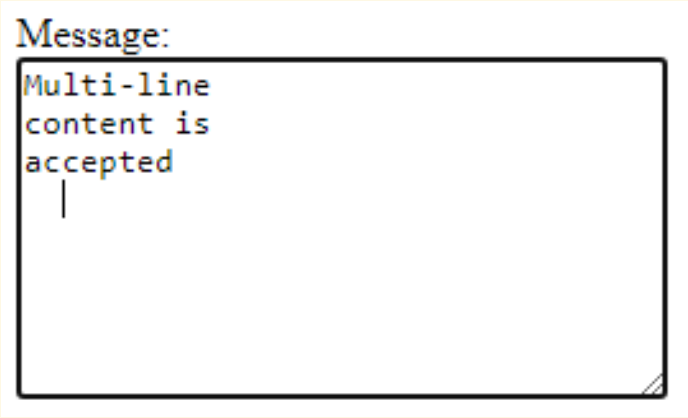
<input> Pseudo-classes

- Useful in styling with CSS, or querying elements with JavaScript
  - `:enabled`, `:disabled`, `:read-only`, `:read-write`, `:checked`, `:required`, `:optional`, `:blank`
  - Common pseudo-classes are also useful: `:hover`, `:focus`, `:active`

- Examples:
  - ```
    input[type=text]:focus:read-only {
      color: red;
    }
    ```
  - ```
    document.querySelector(
       'input[name="gender"]:checked').value
    ```

<textarea>

- A control that accepts multi-line text input
- Example:
  - ```
    <label for="msg">Message:</label><br/>
    <textarea id="msg" name="msg" rows="5"
         minlength="100" maxlength="2000">
    Multi-line
    content is
    accepted
    </textarea>
    ```



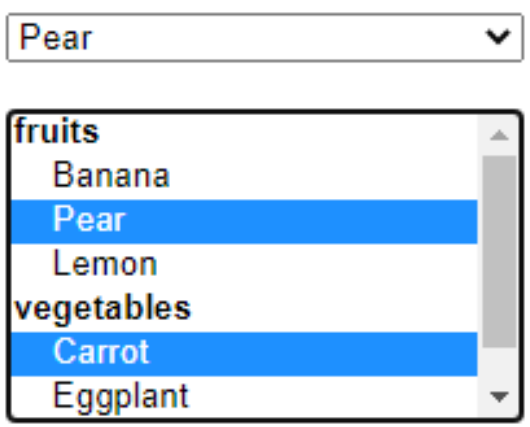
  - Getting value with JavaScript:
    ```
    document.getElementById("#msg").value
    ```
    ```

<button>

- 3 types: `button` (default), `reset`, `submit`
- Like `<input>` with same types (`button`, `reset`, `submit`) but more advanced:
  - Easier to style with CSS
  - Accepts HTML content
- Examples:
  - `<button type="submit">Let's go</button>`
  -
```
<button>
  <img src="save.gif" alt="Save icon"/><br/>
  <strong>Save</strong>
</button>
```

<select>

- Control allowing user to choose one or more of the given options
  - Attn: Styling <select> with CSS is very challenging



- Examples:
  - ```
    <select id="simple" name="simple">
     <option value="banana">Banana</option>
     <option value="pear" selected>Pear</option>
     <option value="lemon">Lemon</option>
    </select>
    ```
  - ```
    <select id="multi" name="multi" multiple size="7">
     ...
    </select>
    ```
  - Get selected options and values:

```
const simpleVal = document.getElementById('simple').value;

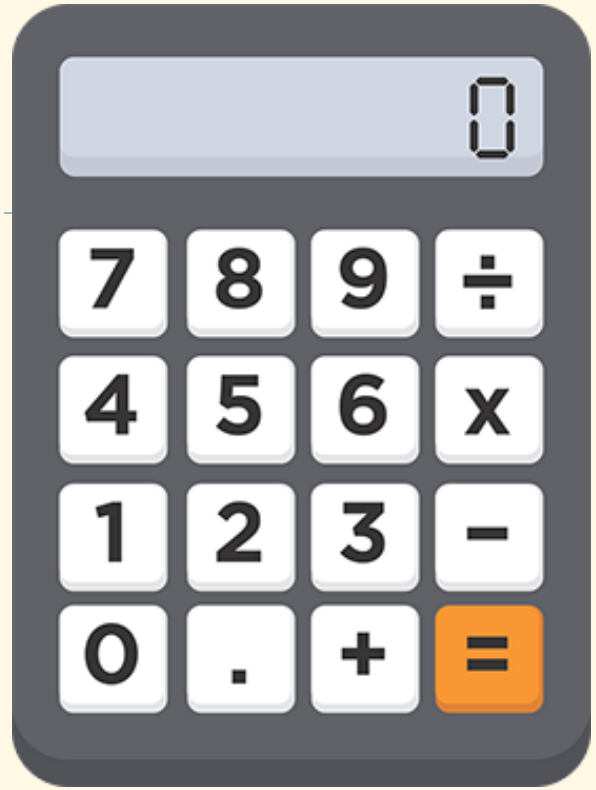
const sel = document.getElementById('multi').selectedOptions;
const multiVal = Array.from(sel).map(e => e.value);
```

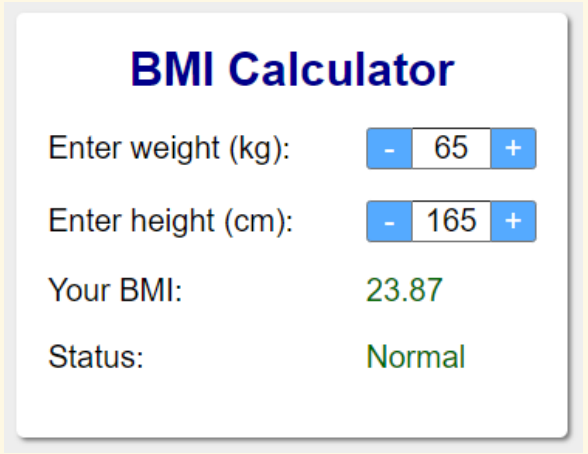
Form-Related Events

- `focus`, `blur`: control gets/loses focus
- `input` (on `<input>`, `<textarea>`): control's value is changed
- `change` (on `<input>`, `<select>`, `<textarea>`): control's value is committed by user
- `click`: button, checkbox being clicked
- `select`: some text has been selected in a control
- `reset`: form's reset button is clicked
- `submit`: form data being submitted by user

Exercises

1. Create a simple calculator as in the figure
2. Create a BMI calculator as in the figure
   - $BMI = w/h^2$ (w in kg, h in meters)
     - < 18.5 : underweight
     - 18.5 ÷ 25 : normal
     - 25 ÷ 30 : overweight
     - ≥ 30 : obese





Web UI Frameworks

Why is a Framework Important?

- Web UI development has fun, but it's laborious to maintain
  - The responsiveness to wide range of user devices
  - The compatibility among web browsers (and their versions)
  - The consistency in look-and-feel across all site's pages
- Most popular UI Frameworks:
  - Tailwind: https://tailwindcss.com/
  - Bootstrap: https://getbootstrap.com/
  - Angular: https://angular.io/
  - Foundation: https://get.foundation/
  - Bulma: https://bulma.io/
  - Materialize: https://materializecss.com/